

Design of a Four-Bit Binary to Decimal Conversion

A Module 2 View: Combinational Logic — K-Maps, Decoders and the Road to an ALU

Our converter is not just a class project — it's a textbook example of a decoder, one of the core combinational building blocks this module introduces.

0

Memory elements

20

Gates total

16

Parallel comparisons



Where This Project Fits in Module 2

1 Combinational logic

Our converter's output depends only on the current 4-bit input — the defining idea of this whole module.

2 K-maps

We check whether our design can be simplified further — and see why a one-hot decoder is a special case.

3 Decoders

Our project literally is a decoder — one of the data-routing blocks this module names directly.

4 Toward the ALU

Decoders, muxes, adders and comparators are exactly the pieces the module combines into an ALU.

Combinational Logic: No Memory, Just Now

- A combinational circuit's output depends only on its current inputs — never on what happened before.
- Our converter is a clean example: feed it 1101 today or feed it 1101 tomorrow — output Y13 goes HIGH either time, with no memory of past inputs.
- This is what makes it purely gate-built, with no clock and no stored state — unlike the registers and memory covered in later modules.

inputs → output

no clock
no memory
no history

same input, right now → same output, right now

Truth Table Recap

All 16 combinations of the 4-bit input ABCD and their decimal output

ABCD	Y	ABCD	Y	ABCD	Y	ABCD	Y
0000	0	0100	4	1000	8	1100	12
0001	1	0101	5	1001	9	1101	13
0010	2	0110	6	1010	10	1110	14
0011	3	0111	7	1011	11	1111	15

Simplifying With K-Maps

- A Karnaugh map (K-map) groups adjacent 1s in a truth table so fewer, larger gate groups can replace many small ones — fewer gates, faster and cheaper hardware.
- For our converter, each output (like Y5) is HIGH for exactly one input row — a single 1 sitting alone on its K-map, with no adjacent 1 to group it with.
- So $Y5 = A'BC'D$ is already as simple as it gets: one minterm, one AND gate. K-maps matter most when an output is HIGH for several rows at once — which happens elsewhere in a real ALU, just not in a one-hot decoder.

One isolated 1 on a K-map → no simplification possible → $Y5 = A'BC'D$ stays exactly as written.

Decoders: What Our Project Really Is

- A decoder takes an n -bit code and activates exactly one of 2^n output lines — the one matching that code.
- Our converter is precisely a 4-to-16 decoder: 4 input bits select exactly one of 16 decimal output lines.
- Decoders show up everywhere in a real CPU: selecting a memory address, picking which register to write, or turning an opcode into the one control signal that matters.

4-to-16

DECODER

4 inputs → 16 outputs
exactly one HIGH at a time

Decoder vs Multiplexer

Two small, everywhere-used blocks that move data in opposite directions.

DECODER (our project)

One code in → many lines out. Exactly one of 16 outputs goes HIGH, chosen by the 4-bit input.

one → many

MULTIPLEXER

Many lines in → one out. A select input chooses which single input value is passed through.

many → one

A Decoder Is 16 Comparators in One

- A comparator answers one question: “does this input equal that value?”
- Our AND-gate for Y5 is exactly that question, hardwired: “is the input equal to 0101?” — nothing more.
- Our whole converter is 16 of these equality tests running side by side, all sharing the same 4 inputs — one comparator per possible decimal value.

$Y5 = A'BC'D$ is really “input == 0101 ?”

Arithmetic Circuits We're Not Using — Yet

Half Adder

Adds two single bits, producing a sum and a carry out.

Full Adder

Adds two bits plus a carry in — the building block chained into wide adders.

Ripple-Carry Adder

Chains full adders together; wider adders are slower because the carry has to ripple through.

Our converter decodes an existing value — it doesn't add anything, so none of these appear in its circuit. They matter for this module because a real ALU needs them alongside decoders like ours.

Subtraction & Two's Complement

- An adder becomes an adder/subtractor by inverting one input and feeding a control bit into the carry-in — turning “+B” into “+(-B)” using two's complement from Module 1.
- Our project has no subtractor: it only decodes an unsigned 4-bit value that already exists, so there's nothing to add or subtract.
- This is exactly why Module 1 kept our values unsigned (0–15) — two's complement becomes relevant only once subtraction enters the picture, later in a full ALU.

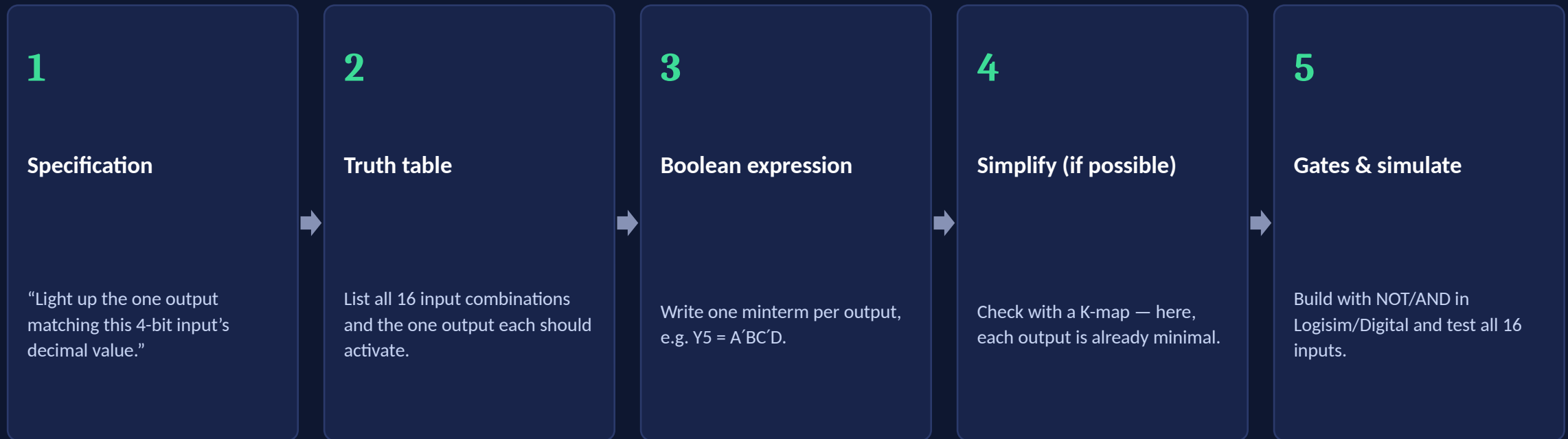
From Our Decoder to a Full ALU

An ALU is these same small blocks, combined with an operation-select input.



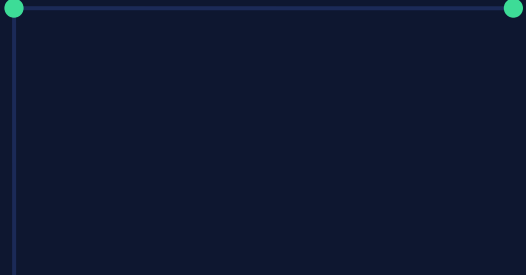
An operation-select input picks which of these results actually reaches the output — our converter shows exactly how that final routing step works.

Designing a Combinational Circuit: The Flow

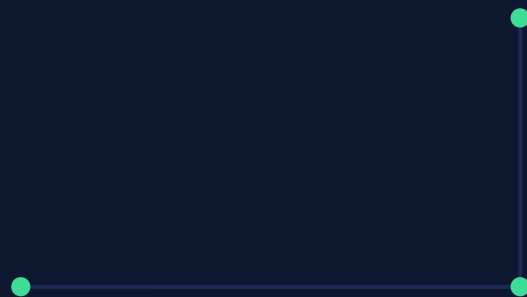


Module 2 Takeaways

- Our converter is a pure combinational circuit — its output depends only on the current 4-bit input, with no memory involved.
- It's also a real-world example of a decoder, one of Module 2's core data-routing blocks, and can be read as 16 parallel comparators.
- K-maps confirm each output is already minimal — a useful special case, since a one-hot decoder can't be simplified further.
- Decoders, muxes, adders and comparators like these are exactly what combine into a full ALU, this module's milestone.



Thank You



Design of a Four-Bit Binary to Decimal Conversion — Module 2: Combinational Logic & the ALU